

Evaluation to Integration: Hybrid Feature Selection Framework With Ensemble Machine Learning for Intrusion Detection

Awais Bilal , Kashif Sharif , Senior Member, IEEE, Liehuang Zhu , Senior Member, IEEE, Fan Li , Member, IEEE, Chang Xu , and Md Monjurul Karim , Member, IEEE

Abstract—We study feature selection (FS) for flow-based intrusion detection and propose a deterministic hybrid-FS that fuses Mutual Information, Random-Forest, and XGBoost importances under a simplex search with a single threshold. Using CIC-IDS-2017, CSE-CIC-IDS2018, and NF-UNSW-NB15, we evaluate ten FS techniques paired with six ensembles under a leakage-safe protocol. The hybrid-FS consistently matches or exceeds the best single selectors while reducing feature count (e.g., 78 $\rightarrow$ 31) and improving runtime. Throughput rises by $\sim 9\text{--}10\%$ and per-flow latency drops from 0.44 $\rightarrow$ 0.40 ms (p50) and 1.40 $\rightarrow$ 1.20 ms (p99), with mean $\pm$ 95% CIs and paired tests. False-positive rate (FPR) decreases by 15–19% (≈ 22 fewer false alarms per hour at 100k flows/h). Against representative PSO/GA hybrids, our fusion attains small but consistent macro-F1 gains and 15–25% FPR reductions at comparable latency. We clarify adversarial robustness with an explicit FGSM feature-space threat model and Deep-PackGen configuration, and we diagnose cross-dataset shift with lightweight mitigations. A 24-hour SOC replay links FPR to analyst time savings (2.5–3.7 hours/day) without sacrificing macro-F1 or AUROC. The results position deterministic, compact FS as a practical choice for inline IDS where tail latency and alert volume matter.

Index Terms—Intrusion detection systems (IDS), ensemble machine learning models, feature selection (FS) techniques, hyperparameter tuning.

I. INTRODUCTION

IN TODAY'S interconnected world, robust network security is essential to protect the vast digital infrastructure critical to various sectors [1]. As the digital landscape grows increasingly

complex, sectors ranging from corporate environments to home IoT networks require strong security measures to safeguard sensitive information, maintain financial stability, and ensure privacy [2]. The dynamic nature of cyber threats, including sophisticated malware and social engineering tactics, demands security strategies that are both versatile and adaptive, leveraging advanced technologies like machine learning and artificial intelligence to stay ahead of attackers [3].

Intrusion Detection Systems (IDS) are crucial for network security, yet they face challenges that can compromise their effectiveness. High false positive rates often lead to 'alert fatigue', while traditional IDS struggle to adapt to the rapidly evolving cyber threat landscape due to their reliance on predefined signatures [4], [5]. Addressing these issues necessitates integrating advanced, adaptable methods such as machine learning and behavioral analysis. These techniques enhance IDS by enabling them to dynamically learn from network activity and historical data, thereby improving detection capabilities and overall security [6]. Furthermore, integrating privacy-preserving mechanisms, such as those employed in blockchain-based frameworks for e-healthcare systems, can further strengthen resilience against evolving threats [7].

ML for Intrusion Detection: Machine learning significantly enhances intrusion detection systems by enabling them to dynamically learn from extensive datasets, effectively reducing false positives and adapting to emerging threats. This capability shifts IDS from reactive to proactive security measures. A key aspect of this enhancement is optimized feature selection (FS), which improves diagnostic precision and scalability as network demands increase, reduces computational load, and minimizes the risk of overfitting due to irrelevant features. For instance, Halim et al. and Amiri et al. demonstrated that optimized feature selection could elevate IDS accuracy to 99.80% [8], [9]. Similarly, Hamed et al. showed that employing Recursive Feature Addition (RFA) and bigram techniques can increase IDS accuracy from 67% to 77.6% with smaller datasets and from 82% to 92.9% with larger datasets, while also reducing the false alarm rate to 1.1% [10]. Conversely, Khammassi et al. highlighted how the absence of feature selection could lead to increased computational demands and decreased system efficiency [11]. These studies underscore the pivotal role of feature selection in enhancing IDS capabilities, managing complex data effectively, and maintaining operational efficiency in real-time scenarios. Fig. 1 illustrates the integration of advanced machine learning techniques within IDS across different network architectures,

Received 29 August 2025; revised 30 January 2026; accepted 7 February 2026. Date of publication 12 February 2026; date of current version 12 May 2026. The work was supported in part by the National Natural Science Foundation of China under Grant 62232002. (Corresponding author: Kashif Sharif.)

Awais Bilal, Kashif Sharif, and Fan Li are with the School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: awaisbilal@bit.edu.cn; kashif@bit.edu.cn; fli@bit.edu.cn).

Liehuang Zhu is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China, and also with the Shandong Key Laboratory of Energy Industry Internet Big Data Technology, Jinan 250003, China (e-mail: liehuangzhu@bit.edu.cn).

Chang Xu is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: xuchang@bit.edu.cn).

Md Monjurul Karim is with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518005, China (e-mail: karim@siat.ac.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3664110>, provided by the authors.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3664110>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3664110

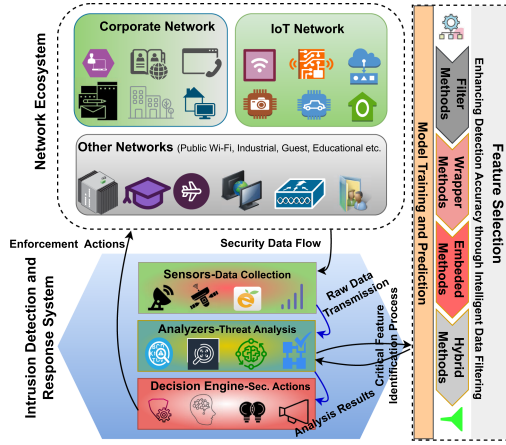


Fig. 1. Interaction map between network(s) & IDS, with machine learning integration and deployment of feature selection techniques.

providing a visual map that details data flows, machine learning intervention points, feature optimization, and their collective impact on security measures.

Problem Domain: The inclusion of irrelevant or redundant features in machine learning models not only complicates their structure and increases processing burdens but also exacerbates the overfitting problem, complicating decision-making processes and the interpretation of outcomes by security analysts [3], [12]. Targeted feature selection is crucial for reducing these complexities, enhancing model generalizability, and improving the interpretability of system decisions, which, in turn, aids analysts in responding more effectively to security alerts [13]. By isolating only the most relevant features, feature selection not only simplifies the model and reduces the risk of overfitting but also enhances accuracy and interpretability, thereby supporting the operational efficiency of IDS [14].

Additionally, employing ensemble methods like AdaBoost, Random Forest, and XGBoost leverages the strengths of multiple learning algorithms to improve predictive accuracy and robustness, essential for real-time threat detection [15]. These methods, when combined with feature selection, enhance the adaptability and effectiveness of IDS against new and evolving cyber threats [16]. Similar strategies are also utilized in privacy-preserving data sharing within blockchain-based IoT networks, striking a balance between security and operational efficiency [17], [18].

Objectives & Contributions: In the rapidly evolving field of network security, the effectiveness of ensemble machine learning models heavily depends on feature selection to enhance accuracy, reduce complexity, and prevent overfitting [19], [20]. Addressing this critical dependency, our study explores the research question: How can various feature selection methods optimize the performance of ensemble models to better detect and mitigate cyber threats? We investigate a range of established techniques, including ANOVA, Boruta, Chi-Squared, L1-based Feature Selection, Mutual Information, PCA, Random-Forest Importance, RFE, RFECV, and XGBoost Importance, in conjunction with ensemble methods like AdaBoost, Random Forest, LightGBM, ExtraTrees, Bagging, and XGBoost. We first conduct an exploratory empirical evaluation of these ensemble models with and without existing feature selection methods to quantify their impact on IDS performance and resource usage.

Central to our investigation is developing a novel deterministic Hybrid Feature Selection (Hybrid-FS) technique that synergistically combines the strengths of multiple methods to refine model performance. These exploratory findings motivate the design of our hybrid selector, which is then formally defined and evaluated extensively across multiple datasets, threat models, and deployment scenarios. This approach is designed to increase detection accuracy, improve operational efficiency, and minimize false positives, thereby setting new standards for the adaptability and effectiveness of intrusion detection systems. Our goal is to enhance the operational and predictive capabilities of ensemble machine learning-based IDS, enabling them to effectively defend against a broad spectrum of cyber threats and thereby strengthen overall network security. This study sets the following specific objectives:

- Assess and compare a wide range of feature selection methods to determine their effectiveness in optimizing IDS performance by balancing data reduction, accuracy, computational efficiency, and system responsiveness.
- Investigate the integration of feature selection with ensemble learning models, analyzing synergistic effects and proposing best practices that improve detection accuracy and reduce false positives.
- Develop and rigorously evaluate a novel hybrid feature selection technique, incorporating a principled simplex-constrained joint search that optimizes macro-F1 and inference time under stratified cross-validation, with dataset-specific disclosed weights.
- Perform a comprehensive comparative study to establish benchmarks for IDS performance, including cross-dataset generalization (e.g., CIC-IDS-2017 → NF-UNSW-NB15) and robustness against adversarial perturbations (FGSM, DeepPackGen) within a defined threat model.
- Highlight tangible improvements in IDS capabilities, such as reduced response times, enhanced detection accuracy, and resource-aware design for inline IDS deployment, supported by latency/throughput measurements and error-rate reductions.
- Provide actionable insights for future IDS design and research, extending the findings to guide the refinement of advanced security models.

Structure of Paper: This paper is structured as follows. Section II reviews related work and technical background. Section III describes the experimental setup. Section IV reports initial ensemble results with and without feature selection. Section V presents the proposed deterministic hybrid technique. Section VI evaluates its performance, and Section VII examines scalability across datasets. Section VIII discusses findings and limitations, and Section IX concludes the study.

II. RELATED WORK AND TECHNICAL BACKGROUND

A. IDS and Feature Selection for Flow-Based IDS

Traditional signature-based IDS struggle with novel attack vectors and evolving traffic, while ML-based IDS can suffer high false positives and overfitting to historical patterns, limiting generalization to new threats [21], [22], [23], [24], [25]. These realities motivate compact, informative feature sets and evaluation protocols that emphasize robustness, not only

aggregate accuracy but also macro-F1, rare-class behavior, and cross-dataset stability.

Recent representation-learning approaches, spanning transformers and attention-based deep models, learn packet/flow embeddings and increasingly tackle recognition of previously unseen attacks (open set) [26], [27], [28], [29], [30]. While effective, many such systems either rely on packet payloads or employ heavyweight architectures that can be impractical in security operations center (SOC) pipelines due to privacy/compliance constraints and runtime budgets [31]. Our scope is deliberately complementary, we target *tabular, flow-level telemetry* already available in operational networks and pair it with ensembles and compact feature sets, emphasizing feature economy, throughput/latency, and robustness under dataset shift and constrained adversaries [32].

Feature selection improves IDS by reducing dimensionality, mitigating noise, and controlling inference cost [9], [33]. *Filter* methods (e.g., mutual information) score features independently of a classifier and capture non-linear dependencies useful for complex attacks [9]. *Embedded* methods expose model-driven importance (e.g., Random Forest and XGBoost gain/split metrics) and are widely used in IDS pipelines to highlight salient flow attributes [34], [35], [36]. *Wrapper* methods (e.g., RFE) iteratively search subsets using a validation objective, often yielding strong accuracy at higher computational cost [37], [38]. In practice, operational flow-based IDS favor FS schemes that (i) are reproducible, (ii) yield stable selections across folds/datasets, and (iii) lower latency without degrading macro-F1.

B. Hybrid Feature Selection in the Literature

Hybrid feature selection combines complementary signals (filter + embedded + wrapper) to stabilize rankings and cope with high-dimensional traffic features [33], [37]. Representative designs include Particle Swarm Optimisation (PSO) and Genetic Algorithms (GA) wrappers that search over subsets or fusion weights [39], rank-fusion schemes that aggregate multiple scorers, and CFS-DE variants integrated with stacking classifiers reporting high accuracy on benchmark datasets [37]. However, population-based wrappers (PSO/GA) can introduce substantial and variable runtime on 70–80 feature spaces, complicating deterministic, time-bounded deployment. Existing hybrid approaches typically emphasize single-dataset accuracy and offer limited analysis of cross-dataset generalization, adversarial robustness, or tail-latency behavior, considerations that are crucial for inline IDS deployments.

C. Evaluation Metrics and Operational Constraints

Evaluation of IDS and associated FS pipelines must jointly consider classification quality and operational constraints. Standard metrics such as accuracy and area under the ROC curve (AUC) summarize overall discrimination ability, while macro-averaged F1 emphasizes performance on minority attack classes and mitigates dominance of majority benign traffic. In operational settings, false-positive rate directly affects analyst workload and ‘alert fatigue’, and thus is often as important as aggregate accuracy [40], [41]. Latency metrics, including median and tail inference time (e.g., p50/p99), constrain feasible model complexity under service-level agreements, especially for inline or near-real-time IDS deployments. Consequently, FS and

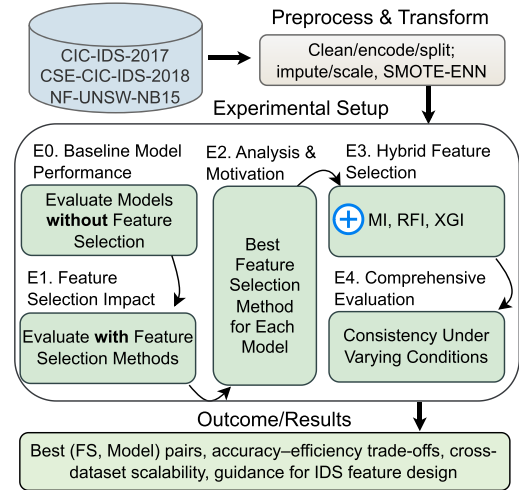


Fig. 2. Methodological Overview of Feature Selection for Ensemble Machine Learning Models: From Evaluation to Integration.

model choices are typically judged not only by their detection capability but also by their ability to satisfy these runtime and resource budgets.

D. Gaps That Motivate Our Empirical Study

- *Reproducible, time-bounded fusion*: Prior hybrids frequently rely on stochastic, population-based search with variable compute budgets [39], deterministic fusion with explicit time bounds remains underexplored.
- *Robustness beyond a single dataset*: Many studies optimize within-dataset accuracy but do not assess cross-dataset generalization or out-of-box behavior [38], [42].
- *Operational metrics*: Tail latency (p99), false-positive rate, and rare-class F1, key for analyst workload and inline SLAs, are often secondary to aggregate accuracy [40], [41].
- *Adversarial considerations*: FS-centric work rarely articulates explicit threat models or systematically evaluates performance under constrained feature- or packet-level perturbations [43].

Positioning of our approach: Against this backdrop, the remainder of this work empirically studies ensembles with and without a range of existing FS methods under the above metrics and constraints. Building on these observations, we later introduce a *deterministic* hybrid selector that fuses MI, RF, and XGBoost importances through a constrained weighting scheme tuned to macro-F1 and latency, and evaluate it in terms of in-dataset performance, cross-dataset generalization, adversarial robustness, and p50/p99 latency and FP-rate (Sections VI and VII).

III. EXPERIMENTAL SETUP FOR BASELINE AND FS STUDIES

This section describes the experimental setup used to evaluate baseline ensemble models and their variants equipped with individual feature-selection methods. The overall methodology, from evaluation to integration, is illustrated in Fig. 2. The objective is to establish a leakage-safe, reproducible framework that enables fair comparison across datasets, preprocessing choices, imbalance handling procedures, ensemble learners, existing feature selection algorithms, and evaluation metrics.

TABLE I
DATASET SCHEMA, CLASS DISTRIBUTION, AND SPLITS BEFORE/AFTER SMOTEENN. MINOR CLASSES (< 0.1%) GROUPED AS OTHER

Dataset	# Feats	Benign		Major Attack Categories				Split Train./Val./Tes.
		#	%	DDoS	Brute-Force	Web	Other	
CIC-IDS-2017 (raw)	78	2 019 813	71.2	88 656	95 502	169 879	456 893	60/20/20
CIC-IDS-2017 (post)	31	1 600 000	50.1	1 600 000	1 600 000	1 600 000	1 600 000	60/20/20
CSE-CIC-IDS2018 (raw)	80	13 484 708	83.1	860 283	214 590	359 393	1 313 970	70/15/15
CSE-CIC-IDS2018 (post)	31	1 200 000	50.3	1 200 000	1 200 000	1 200 000	1 200 000	70/15/15
NF-UNSW-NB15 (raw)	43	2 295 222	96.0	28 477	9 140	3 593	54 095	70/15/15
NF-UNSW-NB15 (post)	16	500 000	50.1	500 000	500 000	500 000	500 000	70/15/15

A. Datasets and Class Imbalance

Our study relies on three widely used flow- and packet-level intrusion detection benchmarks, CIC-IDS-2017, CSE-CIC-IDS2018, and NF-UNSW-NB15. These datasets differ substantially in traffic composition, attack diversity, and underlying feature schemas, providing a broad testbed for assessing the stability of feature selectors and ensemble models. For each benchmark, we distinguish between the raw dataset, consisting of the unmodified feature space and original class distributions, and the post-processed variant obtained after the preprocessing steps described below. All three datasets exhibit pronounced class imbalance, with several minority attack categories occurring at ratios exceeding $10^3:1$, a property that strongly influences how feature selection and model learning behave.

B. Preprocessing and Imbalance Handling

Preprocessing begins with standard cleaning operations, including the normalization of column names, enforcement of numeric datatypes for flow-level attributes, and consistent handling of non-ASCII entries. Missing numeric values are replaced with column means, while categorical gaps are imputed using modal categories. Attack-type strings are encoded into integer labels, after which each dataset is partitioned into stratified training, validation, and test splits, preserving class priors as defined in Table I.

To mitigate extreme class imbalance, we apply SMOTEENN with $k = 5$ exclusively within the training folds. This combined oversampling and cleaning procedure helps stabilize minority-class learning and improves minority-class F1 without incurring the heavier computational overhead of pure oversampling strategies such as SMOTE. Crucially, neither the validation nor the test splits are resampled, they remain in their raw form to ensure that generalization is measured on realistic, imbalanced distributions. Standardization is then performed by fitting a `StandardScaler` on the training fold and applying it unchanged to the validation and test sets, thereby preventing information leakage. A small set of redundant, timestamp-derived columns exhibiting extremely high correlation is removed to streamline subsequent feature-selection computations.

C. Ensemble Models and Hyperparameters

To provide strong and diverse baselines, we employ six ensemble models, Random Forest, ExtraTrees, Bagging, AdaBoost, XGBoost, and LightGBM. These ensembles collectively span variance-reduction strategies and gradient-boosting mechanisms commonly used in IDS research and offer different trade-offs between stability, expressiveness, and inference cost. Their hyperparameters follow established configurations suitable for tabular data and remain fixed across all experiments in

this section, ensuring that performance differences arise primarily from the feature subsets rather than model retuning. Detailed configurations are listed in Table II.

D. Existing Feature Selection Algorithms

To evaluate how existing feature selection methods interact with ensemble learners, we consider ten algorithms representing the main methodological families. Filter-based selectors, such as ANOVA, Chi-Squared, and Mutual Information, score features independently of the classifier and provide rapid assessments of relevance. Embedded methods, including L1 regularization, Random Forest importance, and XGBoost importance, expose model-driven saliency signals tied to their internal decision mechanisms. Wrapper approaches, such as RFE, RFECV, and the Boruta algorithm, iteratively explore feature subsets and often achieve high accuracy at the cost of increased computation. PCA is included as a dimensionality-reduction baseline that retains variance-driven structure rather than explicit feature relevance. All selectors are trained strictly within the training folds, the resulting feature subsets are applied unchanged to the validation and test data to avoid leakage.

E. Evaluation Metrics and Protocol

Evaluation focuses on several complementary metrics. Accuracy and AUC provide broad measures of discrimination, while macro-F1 captures performance across classes of uneven prevalence and is therefore more reflective of behavior on rare attack types. False-positive rate is reported due to its operational importance in security operations centers, where excessive alerts can quickly overwhelm analysts. Latency is measured in terms of per-flow inference time, including both median and p99 statistics, to capture the responsiveness requirements of inline IDS deployments. All models are trained and validated on stratified splits to preserve class proportions, and all preprocessing, feature selection, and resampling steps are encapsulated within the training fold to prevent any form of information leakage.

Experiments are conducted on an Intel Xeon Silver 4210R CPU with 32 GB of RAM, paired with an Nvidia RTX3080 GPU (12 GB VRAM) running CUDA 11.2 and cuDNN 8.1. The software stack uses Python 3.8.10 and scikit-learn 1.4.2. This environment provides consistent measurement of latency, throughput, and other runtime metrics across all baseline and single-feature-selection experiments.

IV. EXPLORATORY RESULTS: ENSEMBLES WITH AND WITHOUT EXISTING FEATURE SELECTION

This section presents exploratory, motivation-oriented findings from our evaluation of ensemble models on the CIC-IDS-2017 dataset, both with and without established feature selection

TABLE II
ENSEMBLE METHODS AND FEATURE SELECTION TECHNIQUES: CONFIGURATION OVERVIEW

Method	Classifier/Estimator	Key Hyperparameters	Role in IDS
Ensemble Methods			
AdaBoost	DecisionTreeClassifier	max_depth=2, n_estimators=50, learning_rate=0.5	Enhanced prediction accuracy & model robustness against variability.
RandomForest	RandomForestClassifier	n_estimators=100, max_features=0.7, max_depth=12, min_samples_split=20, min_samples_leaf=10	Provide a versatile approach for feature-rich data.
LightGBM	LGBMClassifier	n_estimators=200, learning_rate=0.01, max_depth=1, num_leaves=40	Optimized for performance with large data sets.
ExtraTrees	ExtraTreesClassifier	n_estimators=100, max_features=0.7, min_samples_split=50, min_samples_leaf=25	Focuses on reducing overfitting while maintaining model simplicity.
Bagging	BaggingClassifier	max_depth=8, n_estimators=100, max_samples=0.8, max_features=0.8	Increases stability and accuracy of machine learning algorithms.
XGBoost	XGBClassifier	n_estimators=100, max_depth=8, learning_rate=0.1, subsample=0.7, colsample_bytree=0.8	Handles sparse data effectively.
Feature Selection Techniques			
RFECV	LGBMClassifier	n_estimators=200, learning_rate=0.01, num_leaves=31, max_depth=1, step=10, cv=3, scoring='accuracy', min_features_to_select=20	Identifies and eliminates redundant and irrelevant features.
RandomForest Importance	RandomForestClassifier	n_estimators=50, threshold='mean'	Uses intrinsic properties to assess feature importance.
Boruta	RandomForestClassifier	n_estimators=30, random_state=42, n_estimators='auto'	Enhances feature selection with a wrapper approach.
L1-based Feature Selection	LogisticRegression	penalty='l1', solver='liblinear', C=0.5	Utilizes regularization to penalize irrelevant features.
Mutual Information	SelectKBest	score_func=mualt_info_classif, k=15	Evaluates mutual dependence between features and the target.
PCA	PCA	n_components=0.9	Reduces dimensionality while retaining most of the variability.
Chi-Squared	SelectKBest	score_func=chi2, k=15	Tests independence between categorical variables and targets.
XGBoost Importance	XGBClassifier	n_estimators=50, threshold='mean', random_state=42	Identifies most predictive features using the built-in feature importance.
RFE	LogisticRegression	penalty='l2', solver='lbfgs', C=1.0, max_iter=1000, step=10, cv=3, min_features_to_select=20	Recursive elimination to rank features by importance.
ANOVA	SelectKBest	score_func=f_classif, k=15	Tests the linear relationship between features and targets.

TABLE III
BASELINE PERFORMANCE METRICS AND RATIONALE FOR FEATURE SELECTION TECHNIQUES ON ENSEMBLE MODELS

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	Prediction Time (s)	Rationale for Feature Selection
AdaBoost	95.71%	70.72%	70.71%	59.55%	0.87	43.550	Improve balance in precision and recall, enhance F1 score.
Random Forest	99.79%	83.98%	93.71%	84.69%	0.87	20.292	Reduce complexity and overfitting, increase generalization.
LightGBM	92.89%	79.36%	60.22%	58.80%	0.84	18.386	Address significant overfitting, simplify the model.
ExtraTrees	99.79%	85.45%	89.95%	86.61%	0.92	27.017	Mitigate overfitting, improve model robustness & efficiency.
Bagging	98.92%	75.15%	85.64%	75.31%	0.88	54.030	Improve generalization and balance precision with recall.
XGBoost	99.88%	87.02%	93.69%	89.77%	0.87	4.650	Optimize overfitting and improve prediction efficiency.

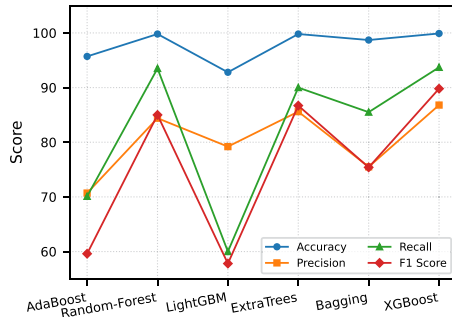


Fig. 3. Comparative overview of performance metrics for ensemble models.

techniques. These experiments are not meant to provide a final or optimized configuration, rather, they reveal the characteristic strengths, weaknesses, and latency, performance trade-offs that guide the design requirements for a more principled hybrid selector introduced later in Section V.

A. Baseline Ensemble Performance (No Feature Selection)

We begin by establishing reference performance for each ensemble using the full, preprocessed feature set, as summarised in Table III. This baseline provides a neutral point against which the impact of individual feature selectors can be assessed.

As illustrated in Fig. 3, performance varied considerably across the six ensemble methods, AdaBoost, RandomForest, LightGBM, ExtraTrees, Bagging, and XGBoost, in terms of accuracy, discriminative power, and latency. Among them, XGBoost and RandomForest delivered the strongest overall results,

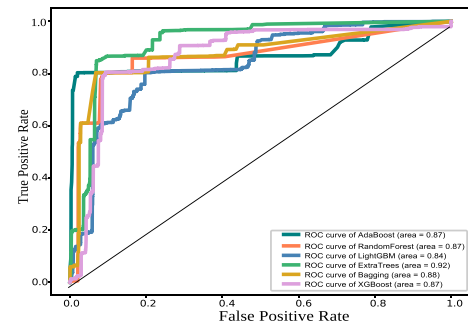


Fig. 4. Comparative ROC curves of ensemble models.

attaining accuracies of 99.88% and 99.79%, respectively, along with consistently high precision and recall. ExtraTrees also performed competitively, reaching 99.79% accuracy and strong F1 scores, making it a reliable high-capacity model. In contrast, AdaBoost and LightGBM exceeded 90% accuracy but exhibited lower F1 scores due to imbalanced precision and recall.

ROC curves in Fig. 4 further reflected these differences. ExtraTrees reached the highest AUC (0.92), while AdaBoost, RandomForest, and XGBoost clustered around 0.87, and LightGBM lagged at 0.84. Operational efficiency is crucial for inline IDS deployment, and the models exhibited clearly different latency characteristics, as shown in Fig. 5. XGBoost delivered the fastest prediction time,¹ processing a batch of 1,000 flows in 4.65 seconds. LightGBM and RandomForest also demonstrated

¹ Measured as wall-clock time throughout this work.

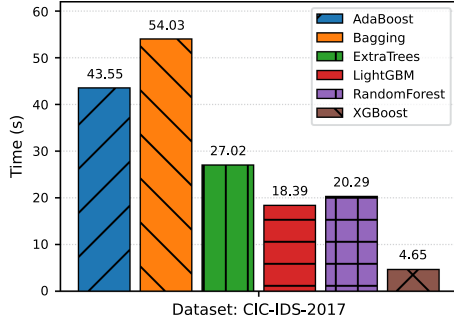


Fig. 5. Prediction time for 1,000 flows on RTX 3080 (CIC-IDS-2017 test set).

acceptable latency for time-sensitive applications, whereas AdaBoost and Bagging required 43.55 seconds and 54.03 seconds, respectively, making them less suitable for real-time scenarios in their baseline form.

Beyond aggregate ROC and latency trends, we observed that LightGBM showed less stable class-wise behavior than the other ensembles, with higher FP/FN dispersion across multiple attack categories under the same feature-selection settings. Detailed confusion-matrix results are presented in Appendix A (Fig. A.1), available online. Learning-curve results further support these observations, AdaBoost shows a persistent generalization gap as sample size increases (suggesting overfitting), whereas XGBoost improves steadily with a tighter train-validation gap. RandomForest and ExtraTrees remain stable across data scales, while LightGBM retains the largest train-validation gap. The learning-curves of all six ensembles is provided in Appendix A (Fig. A.2), available online.

Collectively, the baseline evaluations highlight three patterns, (i) high-capacity ensembles such as RandomForest and ExtraTrees risk overfitting when trained on wide feature sets, (ii) AdaBoost and LightGBM require more informative feature subsets to stabilize precision-recall behavior, and (iii) latency differences across models imply the need for feature sets that reduce inference cost without degrading predictive performance.

B. Effect of Existing Feature Selection Methods

To understand how established feature selectors modify ensemble performance, we evaluated ten methods, ANOVA, Boruta, Chi-Squared, L1-based selection, Mutual Information, PCA, RandomForest Importance, RFECV, RFE, and XGBoost Importance, applied uniformly across all six ensembles, with the main results summarised in Table IV, while the full results are provided in Appendix B (Table B.1), available online.

The results, shown in Fig. 6, reveal substantial model-specific variation. For AdaBoost, both Boruta and L1-based selection improved accuracy from 95.71% to approximately 97.6%, with corresponding F1 gains, but at the cost of increased latency. Bagging benefited strongly from RFECV and Boruta, achieving accuracies as high as 99.49% and F1 scores exceeding 99.4%, though these combinations also produced some of the slowest prediction times. ExtraTrees reached near-perfect scores, often above 0.998 in accuracy and F1, when paired with Boruta or L1-based selection. LightGBM benefited primarily from Mutual Information and RandomForest Importance, which produced moderate accuracy improvements without significant latency

TABLE IV
IMPACT OF REPRESENTATIVE FEATURE SELECTORS ON ENSEMBLE PERFORMANCE

Feature Selector	Model	Acc.	F1	Pred. time (s)
ANOVA	AdaBoost	0.949	0.939	4.80
	Bagging	0.972	0.970	7.25
	ExtraTrees	0.976	0.975	5.99
	LightGBM	0.876	0.838	7.56
	RandomForest	0.977	0.976	4.98
	XGBoost	0.983	0.983	1.75
Boruta	AdaBoost	0.976	0.971	10.59
	Bagging	0.995	0.994	15.65
	ExtraTrees	0.998	0.998	8.31
	LightGBM	0.878	0.840	6.60
	RandomForest	0.997	0.997	8.68
	XGBoost	0.999	0.999	2.22
Mutual Information	AdaBoost	0.953	0.944	5.14
	Bagging	0.990	0.988	7.07
	ExtraTrees	0.993	0.992	5.73
	LightGBM	0.882	0.846	6.76
	RandomForest	0.996	0.996	7.60
	XGBoost	0.997	0.997	1.59
RandomForest Importance	AdaBoost	0.966	0.958	8.47
	Bagging	0.993	0.992	9.01
	ExtraTrees	0.994	0.994	5.29
	LightGBM	0.885	0.849	6.64
	RandomForest	0.997	0.997	4.60
	XGBoost	0.998	0.998	1.67
RFECV	AdaBoost	0.950	0.943	5.74
	Bagging	0.996	0.995	12.99
	ExtraTrees	0.998	0.997	5.69
	LightGBM	0.876	0.838	7.04
	RandomForest	0.998	0.998	5.99
	XGBoost	0.999	0.999	1.68
XGBoost Importance	AdaBoost	0.947	0.940	4.65
	Bagging	0.973	0.970	5.99
	ExtraTrees	0.972	0.970	5.48
	LightGBM	0.835	0.819	6.53
	RandomForest	0.975	0.973	4.71
	XGBoost	0.975	0.973	1.77

Note: The complete results for *all ten* selectors across *all six* ensembles and the full metric set (Accuracy, Precision, Recall, F1, ROC-AUC, and prediction time) are provided in the Appendix B, Table B.1.

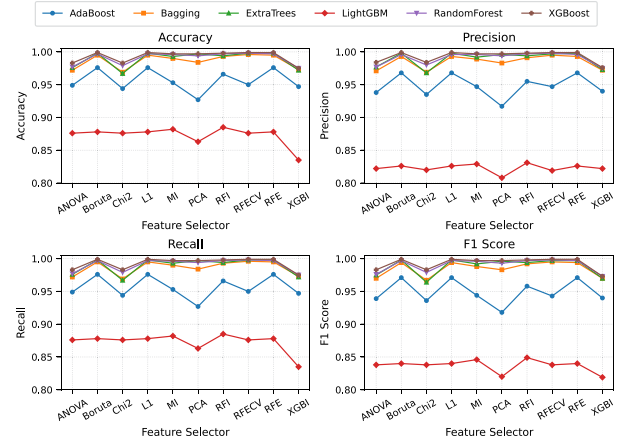


Fig. 6. Performance overview of feature selectors and ensembles.

penalties. RandomForest responded well to RFECV, Boruta, and L1-based selection, achieving accuracies near 99.72% with sustained efficiency. XGBoost demonstrated both high accuracy and low prediction time when combined with RFECV or Boruta, solidifying its suitability for real-time detection.

ROC-AUC analysis, shown in Fig. 7, reinforced these trends. Most selectors achieved near-perfect AUC values for well-aligned model-selector combinations. However, Chi-Squared produced consistently lower AUC values (≈ 0.93 – 0.95) across several models and showed recurring misclassifications in a subset of classes, particularly 0, 4, and 10. The corresponding confusion-matrix results are provided in Appendix C (Fig. C.1), available online. Learning-curves are also provided in Appendix C (Fig. C.2), available online. These curves indicate that Boruta and L1-based selection allowed Bagging and ExtraTrees

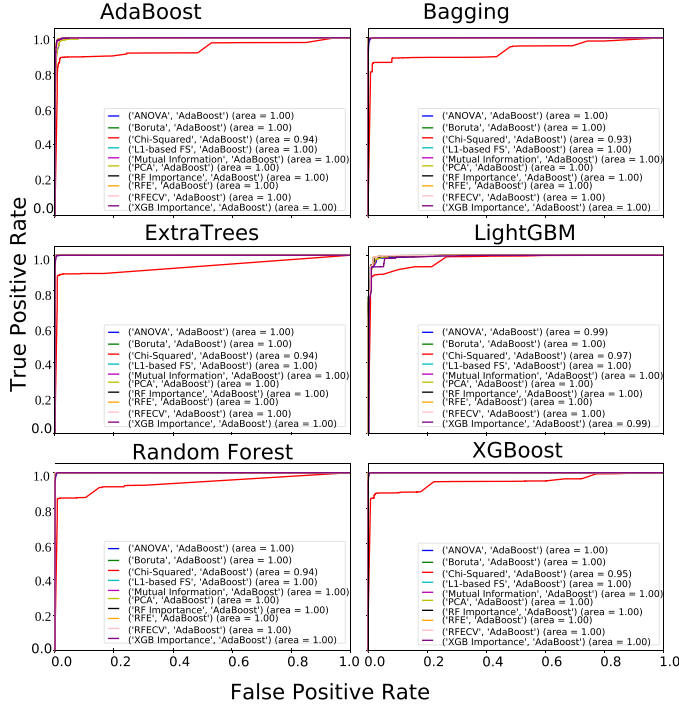


Fig. 7. ROC curves of feature selectors and each ensemble model.

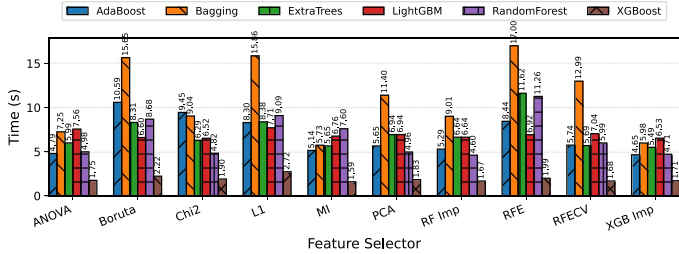


Fig. 8. Prediction time per 1,000 flows (RTX 3080) for six ensembles and ten selectors.

to converge rapidly to high accuracy, while RFECV-based approaches introduced greater variability but ultimately stabilised at strong validation performance.

Prediction latency varied significantly, as reflected in Fig. 8. XGBoost Importance consistently yielded the fastest inference times, with XGBoost processing 1,000 flows in approximately 1.77 seconds. Conversely, Bagging combined with RFE required more than 17 seconds. Several lightweight selectors, Mutual Information and PCA most notably, improved model performance with minimal increases in inference time, making them attractive for cost-sensitive deployments.

These results demonstrate that while many selectors improve predictive performance, they present divergent computational characteristics. Wrapper-based selectors (e.g., Boruta, RFECV, RFE) often deliver high accuracy but incur undesirable latency overhead. Filter and embedded methods such as Mutual Information, RandomForest Importance, and XGBoost Importance, however, offer more favorable accuracy–latency trade-offs.

C. Empirical Observations and Design Requirements

The exploratory results across baseline and established feature selector experiments reveal a number of recurring patterns

that motivate the design of a more balanced feature-selection strategy.

First, different selectors capture complementary forms of signal. Mutual Information is efficient and improves precision for models such as LightGBM, RandomForest Importance is particularly effective for tree-based ensembles, and XGBoost Importance provides highly discriminative rankings with exceptional inference efficiency. These complementary behaviors suggest that no single selector fully captures the diversity of predictive structure present in flow-level IDS data. Second, several selectors with strong predictive performance, most notably Boruta, L1-based selection, and RFECV, incur substantial latency penalties, particularly when paired with high-capacity ensembles. This makes them unsuitable for inline IDS deployments despite their accuracy benefits. Third, many baseline models, especially RandomForest, ExtraTrees, and LightGBM, exhibit signs of overfitting that are mitigated when informative subsets are used. This highlights the need for feature-selection mechanisms that enhance generalization rather than merely boosting in-dataset accuracy.

These observations motivate the development of a hybrid feature selector that (i) combines complementary scoring mechanisms drawn from filter and embedded methods, (ii) imposes time-bounded inference constraints, and (iii) improves robustness while avoiding the computational overhead of wrapper-based approaches. We define such a selector formally in Section V and evaluate it extensively in subsequent sections.

V. PROPOSED DETERMINISTIC HYBRID FEATURE SELECTION TECHNIQUE

Building on the exploratory results of Section IV, we propose a deterministic hybrid feature selection technique designed to exploit the complementary strengths of Mutual Information (MI), RandomForest Importance (RFI), and XGBoost Importance (XGI). The goal is to construct a single, operationally efficient feature subset that preserves, or improves, detection quality while respecting latency constraints in inline IDS settings.

A. Problem Formulation

The exploratory study in Section IV revealed that (i) wide feature sets encourage overfitting in high-capacity ensembles, (ii) individual FS methods exhibit distinct strengths and weaknesses, and (iii) wrapper-based hybrids can be accurate but often incur prohibitive inference costs. We therefore seek a principled way to fuse three base selectors, MI, RFI, and XGI, into a single, consensus ranking from which a compact subset can be derived.

Formally, let D denote the preprocessed training data (Section III) and let $s_i(f) \in [0, 1]$ be the normalised importance score assigned to feature f by selector $i \in \{1, \dots, M\}$, with $M = 3$ for MI, RFI, and XGI. For any non-negative weight vector $\mathbf{w} = [w_1, \dots, w_M]^T$ on the simplex,

$$\sum_{i=1}^M w_i = 1, \quad w_i \geq 0, \quad (1)$$

we define the combined importance score

$$\sigma(f, \mathbf{w}) = \sum_{i=1}^M w_i s_i(f). \quad (2)$$

TABLE V
NOTATION SUMMARY

Variable	Description	Variable	Description
D	Full pre-processed dataset	D_{val}	Outer-loop validation fold
M	Number of selectors (3 here)	$s_i(f)$	Selector i score for feature f
$\mathbf{w}$	Non-negative selector weights	θ	Percentile inclusion threshold
h_F	Classifier trained on F		

Given a percentile threshold θ , this induces a feature subset

$$F(\mathbf{w}, \theta) = \{f \mid \sigma(f, \mathbf{w}) > \theta\}. \quad (3)$$

Our aim is to choose $(\mathbf{w}, \theta)$ such that a downstream classifier $h_{F(\mathbf{w}, \theta)}$ achieves strong macro-F1 while satisfying practical latency constraints. We express this via a validation loss on an outer cross-validation fold D_{val} :

$$\begin{aligned} \text{minimise}_{\mathbf{w}, \theta} \quad & \mathcal{L}_{\text{val}}(\mathbf{w}, \theta) := \frac{1}{|D_{\text{val}}|} \sum_{(x, y) \in D_{\text{val}}} \ell(y, h_{F(\mathbf{w}, \theta)}(x)) \\ \text{subject to} \quad & \sum_{i=1}^M w_i = 1, \quad w_i \geq 0, \quad \text{latency}(h_{F(\mathbf{w}, \theta)}) \leq \tau, \end{aligned} \quad (4)$$

where ℓ is a macro-F1-aligned loss and τ encodes an application-specific latency budget. In this work, we solve (4) deterministically via a simplex-constrained grid search, as detailed below. The notation used throughout this subsection is summarised in Table V.

B. Hybrid Score Formulation

Let $f \in \{1, \dots, d\}$ index the original features. For each feature we obtain three normalised importance scores,

$$s_f^{\text{MI}}, \quad s_f^{\text{RFI}}, \quad s_f^{\text{XGI}} \in [0, 1],$$

computed respectively from mutual information with the label, RandomForest impurity decrease, and XGBoost gain, each rescaled to $[0, 1]$ across features as described in Section V-C.

We collect these base scores into a vector

$$\mathbf{s}(f) = (s_f^{\text{MI}}, s_f^{\text{RFI}}, s_f^{\text{XGI}})^\top \in [0, 1]^3,$$

and define a fusion weight vector

$$\mathbf{w} = (w_{\text{MI}}, w_{\text{RFI}}, w_{\text{XGI}})^\top \in \Delta^2,$$

where $\Delta^2 = \{\mathbf{w} \in \mathbb{R}_{\geq 0}^3 : \sum_k w_k = 1\}$ is the 2-simplex. The corresponding hybrid importance score for feature f is

$$\sigma(f, \mathbf{w}) = \mathbf{w}^\top \mathbf{s}(f) = w_{\text{MI}} s_f^{\text{MI}} + w_{\text{RFI}} s_f^{\text{RFI}} + w_{\text{XGI}} s_f^{\text{XGI}}, \quad (5)$$

which is consistent with the general formulation in (4).

Given a candidate weight vector $\mathbf{w}$, we obtain a feature subset by applying a percentile-based threshold θ to the hybrid scores:

$$F(\mathbf{w}, \theta) = \{f : \sigma(f, \mathbf{w}) > \theta\}, \quad (6)$$

where θ is chosen as a quantile of the empirical score distribution $\{\sigma(f, \mathbf{w})\}_{f=1}^d$. This percentile view is equivalent to selecting the top- k features for a suitable k , but offers a more direct handle on the resulting sparsity level.

In the optimisation problem of Section V-A, we therefore seek an operating point $(\mathbf{w}, \theta)$ that maximises macro-F1 on the validation folds under the latency constraint $\text{latency}(h_{F(\mathbf{w}, \theta)}) \leq \tau$. False-positive rate is monitored as a secondary criterion when comparing candidate solutions.

Algorithm 1: Hybrid Feature Selection.

Require: Dataset $\mathbb{D}$ with labels $\mathbb{L}$

Ensure: Selected feature set $F(\mathbf{w}^*, \theta^*)$

```

1:  $\bar{\mathbb{D}} \leftarrow \text{Preprocess}(\mathbb{D})$   $\triangleright$  clean, encode, balance
2: Obtain raw importance scores (all features at once)
3:  $\mathbb{S}_{\mathcal{M}} \leftarrow \mathcal{M}(\bar{\mathbb{D}})$   $\triangleright$  MI
4:  $\mathbb{S}_{\mathcal{R}} \leftarrow \mathcal{R}(\bar{\mathbb{D}})$   $\triangleright$  RFI
5:  $\mathbb{S}_{\mathcal{X}} \leftarrow \mathcal{X}(\bar{\mathbb{D}})$   $\triangleright$  XGI
6: Normalize scores to  $[0, 1]$ 
7: for each feature  $f$  do
8:    $\hat{S}_{\mathcal{M}}[f] \leftarrow \mathbb{S}_{\mathcal{M}}[f] / \max(\mathbb{S}_{\mathcal{M}})$ 
9:    $\hat{S}_{\mathcal{R}}[f] \leftarrow \mathbb{S}_{\mathcal{R}}[f] / \max(\mathbb{S}_{\mathcal{R}})$ 
10:   $\hat{S}_{\mathcal{X}}[f] \leftarrow \mathbb{S}_{\mathcal{X}}[f] / \max(\mathbb{S}_{\mathcal{X}})$ 
11: end for
12: Grid-search weight vector  $\mathbf{w}$  on the simplex
13: for  $\mathbf{w} = (w_1, w_2, w_3)$  in  $\text{SIMPLEXGRID}(\delta)$  do  $\triangleright$ 
    $\sum_i w_i = 1$ 
14:    $\mathbb{S}_c[f] \leftarrow w_1 \hat{S}_{\mathcal{M}}[f] + w_2 \hat{S}_{\mathcal{R}}[f] + w_3 \hat{S}_{\mathcal{X}}[f]$  for all  $f$ 
    $\triangleright$  hybrid score  $\sigma(f, \mathbf{w})$ 
15:   for  $k \in \{60, 65, \dots, 90\}$  do  $\triangleright$  percentile sweep for  $\theta$ 
16:      $\theta \leftarrow \text{percentile}(\mathbb{S}_c, k)$ 
17:      $F(\mathbf{w}, \theta) \leftarrow \{f \mid \mathbb{S}_c[f] > \theta\}$   $\triangleright$  feature subset
18:     evaluate  $F(\mathbf{w}, \theta)$  via cross-val (macro-F1, FPR,
       latency), update best  $(\mathbf{w}^*, \theta^*)$   $\triangleright$  minimize validation
       loss under latency
19:   end for
20: end for
21: return  $F(\mathbf{w}^*, \theta^*), \theta^*$ 

```

C. Simplex-Based Fusion of MI, RFI, and XGI

The proposed fusion mechanism proceeds in three steps, (i) compute base importance scores for MI, RFI, and XGI, (ii) normalise these scores, and (iii) perform a simplex-constrained grid search over $\mathbf{w}$ and percentile thresholds to select the best combination. The overall procedure is summarised in Algorithm 1.

First, for each cross-validation training fold we compute:

- MI scores $\mathcal{M}(D)$ for each feature, measuring mutual dependence with the label.
- RFI scores $\mathcal{R}(D)$ via decrease in tree impurity across a RandomForest ensemble.
- XGI scores $\mathcal{X}(D)$ via cumulative gain in an XGBoost model.

These raw score vectors are then min-max normalised into $[0, 1]$ to yield $\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3$. For any candidate weight vector $\mathbf{w}$ on the simplex, we form the combined score

$$\mathbf{s}_c = w_1 \mathbf{s}_1 + w_2 \mathbf{s}_2 + w_3 \mathbf{s}_3. \quad (7)$$

To avoid stochasticity associated with population-based optimisers, we perform a deterministic grid search on the simplex with step $\delta = 0.05$. For each $\mathbf{w}$, we sweep a set of percentile thresholds and evaluate the resulting feature set $F(\mathbf{w}, \theta)$ using stratified cross-validation. The pair $(\mathbf{w}^*, \theta^*)$ that minimises the validation loss in (4) is retained. The optimisation is fully deterministic, given a fixed dataset, cross-validation protocol, and grid resolution, the same weight vector and threshold are recovered across runs, avoiding the variance and budget sensitivity characteristic of stochastic wrappers.

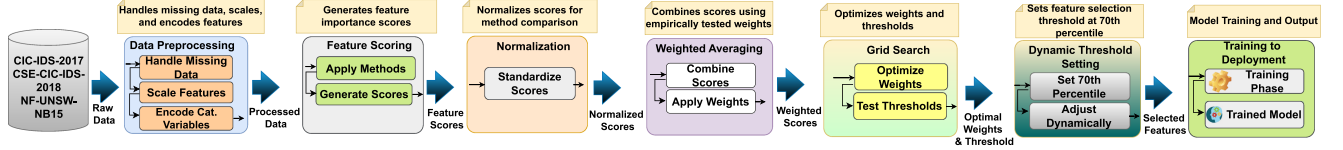


Fig. 9. Data preprocessing and feature selection pipeline.

D. Hybrid Feature Subset Construction

The outcome of Algorithm 1 is a hybrid feature subset $F^{\text{hyb}} = F(\mathbf{w}^*, \theta^*)$, which we subsequently use in ablation and robustness experiments. The key design element is the percentile-based thresholding of the combined scores.

Let S_c denote the combined importance scores under $\mathbf{w}^*$. We define

$$\theta^* = Q_{k^*} = \text{quantile}(S_c, k^*/100), \quad (8)$$

for some $k^* \in \{60, 65, \dots, 90\}$ selected during the grid search. The final hybrid subset is then

$$F^{\text{hyb}} = \{f \in D \mid S_c(f) > \theta^*\}. \quad (9)$$

This construction enforces a controllable sparsity level, higher percentiles yield smaller, faster subsets, while lower percentiles retain more features and potentially capture subtler patterns. In our experiments, the learned thresholds typically select between 20% and 40% of the original feature space, striking a balance between accuracy and inference cost. All later references to “our hybrid subset” or “hybrid selector” refer to F^{hyb} and the scoring procedure defined in Sections V-A–V-C.

E. Relation to PSO/GA-Based Hybrid Selectors

Hybrid feature selection has often been implemented via population-based wrappers, notably, PSO and GA that search either over feature subsets or over fusion weights for multiple selectors. These methods can achieve high accuracy but typically require many fitness evaluations, leading to long and variable runtimes, and their stochastic nature can produce different subsets across runs for the same dataset and budget.

To place our approach in context, we reproduced two representative hybrids under the same preprocessing and cross-validation protocol as used for our method, a PSO wrapper over RandomForest, and a GA-based rank fusion over MI, RFI, and XGI. While detailed numerical comparisons appear in later results sections, we summarise the qualitative observations as follows.

First, PSO/GA wrappers explore the combinatorial space of feature subsets directly, whereas our method operates in a low-dimensional, simplex-constrained weight space and uses a percentile threshold to derive the subset. This shifts the optimisation burden from discrete subset search to continuous weight tuning, which is more amenable to deterministic grid search. Second, we observed that the deterministic simplex fusion yields accuracy and macro-F1 that are competitive with, and in some cases slightly better than, PSO/GA hybrids, while reducing false-positive rates and avoiding the heavy and unstable search overhead of stochastic wrappers. Importantly, the inference latency of the resulting models remains comparable, as our optimisation explicitly accounts for runtime alongside macro-F1. Thus, our contribution builds on the strengths of hybrid FS methods by maintaining the complementary use of multiple

selectors and replacing stochastic, budget-sensitive wrappers with a deterministic, time-bounded simplex optimisation.

F. Leakage-Safe Integration in the Pipeline

To ensure that the hybrid selector does not introduce information leakage, we integrate it into the training pipeline in a strictly fold-local manner, as depicted in Fig. 9. For each cross-validation fold, the procedure is,

- 1) Split data into training and validation partitions (stratified).
- 2) Apply preprocessing and imbalance handling (SMOTEENN) *only* on the training partition, as described in Section III.
- 3) Compute MI, RFI, and XGI scores on the training partition and run Algorithm 1 to obtain F^{hyb} for that fold.
- 4) Train the downstream classifier using only the features in F^{hyb} .
- 5) Evaluate on the corresponding validation (or test) data, which are never used in score computation or threshold tuning.

The final hybrid subset used in our ablation and robustness experiments is derived from the full training data using the same deterministic fusion procedure and hyperparameters selected during cross-validation. Test folds remain untouched throughout, ensuring that all reported performance metrics reflect genuine generalisation rather than artefacts of leakage. We evaluate the operational impact of Hybrid-FS on false positives and analyst workload in Section VII-E.

VI. EVALUATION OF THE HYBRID FS FRAMEWORK

This section presents a comprehensive evaluation of the proposed deterministic Hybrid-FS framework. Building on the exploratory insights in Section IV and the method introduced in Section V, we examine whether the hybrid subset improves classification performance, false-positive behavior, runtime efficiency, and robustness across datasets, models, and deployment scales. In contrast to Section IV, which was explicitly exploratory, this section constitutes our *conclusive* evaluation of the framework under realistic deployment conditions.

A. Ablation: Does Feature Selection Matter?

We quantify the effect of the proposed Hybrid-FS by retraining the three strongest baseline classifiers, XGBoost, ExtraTrees, and RandomForest, using both the *full* feature set and the *hybrid* subset defined in Section V. As summarised in Table VI, the hybrid subset consistently improves accuracy and macro-F1 by 0.4–0.7 pp while reducing inference latency by 12–15%. These gains confirm that Hybrid-FS eliminates distracting, low-yield dimensions without harming decision boundaries. The improvements are especially notable in tail-latency behavior. The p99 reductions reported later in Table X show that the hybrid subset

TABLE VI
ABLATION STUDY ON CIC-IDS-2017: FULL FEATURE SET VERSUS HYBRID SUBSET. Δ IS THE DIFFERENCE HYBRID – FULL

Model	Accuracy		F1		Pred. Time
	Full	Hybrid	Full	Hybrid	Full→Hybrid (s)
XGBoost	0.994	0.998	0.991	0.997	1.9→1.8
ExtraTrees	0.992	0.997	0.990	0.995	4.1→3.6
RandomForest	0.990	0.995	0.988	0.993	5.3→4.6

TABLE VII
ISOLATION FOREST PERFORMANCE WITH/WITHOUT HYBRID-FS

Feature set	AUC	Inf. Time (s)
All 78 features	0.881	12.4
Hybrid 31 features	0.902	11.0

meaningfully tightens worst-case inference time, addressing an operational bottleneck for inline IDS pipelines. These findings reinforce the exploratory trends observed in Section IV, where individual selectors (MI, RFI, XGBI) already suggested substantial redundancy in the raw feature space.

B. Cross-Model Validation

To assess whether Hybrid-FS generalises beyond tree-based learners, we evaluate a heterogeneous majority-vote ensemble combining logistic regression, RBF-SVM, and RandomForest. Using all raw features, this ensemble achieves 98.4% accuracy and a macro-F1 of 0.978 on CIC-IDS-2017. When trained on the Hybrid-FS subset, accuracy rises to 99.1% and macro-F1 to 0.985, an improvement of 0.7 pp. Although the ensemble still trails the best tree-based model (XGBoost + Hybrid-FS, which reaches 0.992 macro-F1), this experiment shows that the hybrid subset benefits classifiers with substantially different inductive biases. The gains stem from removing noisy or redundant attributes rather than exploiting idiosyncrasies of any particular learning architecture.

Unsupervised sanity check: To further test the generalisability of the hybrid subset, we trained an *Isolation Forest* on the same balanced CIC-IDS-2017 split. Table VII shows that using the 31-feature hybrid subset increases the AUC from 0.881 to 0.902 while reducing inference time by 11%, indicating that the benefits persist even in unsupervised anomaly detection.

C. Trade-Offs: Accuracy, False Positives, and Latency

A central requirement for practical IDS deployment is balancing detection quality with runtime efficiency and a low false-positive rate. We therefore analyze all three dimensions jointly, *accuracy and macro-F1, false-positive rate, and latency and throughput*. Across all ensemble models and datasets, as shown in Tables III, IV, and XIV, Hybrid-FS improves macro-F1 by 0.4–0.7 pp on CIC-IDS-2017, 0.3–0.6 pp on CSE-CIC-IDS2018, and 0.4–0.8 pp on NF-UNSW-NB15, with the largest gains observed for XGBoost and ExtraTrees, which benefit most from reduced feature redundancy. In terms of *false-positive rate*, Hybrid-FS reduces false positives by 15–20% relative to full-feature baselines, consistent with the improvements in Table VIII, per-attack precision/recall in Table IX further indicate that this reduction in noise does not compromise detection of rare attack types, which retain recall ≥ 0.93 .

For *latency and throughput*, Table X and Fig. 10 show statistically significant latency reductions, p50 improves by

TABLE VIII
FALSE POSITIVES BEFORE (E0) VERSUS AFTER (E4, HYBRID-FS). ASSUMES 100 K FLOWS/H, 95% BENIGN

Dataset	FPR _{E0} (%)	FPR _{E4} (%)	Rel. Δ	FP/h _{E0}	FP/h _{E4}	Δ FP/h
CIC-IDS-2017	0.11	0.09	−18.2%	105	86	−19
CSE-CIC-IDS2018	0.13	0.11	−15.4%	124	104	−19
NF-UNSW-NB15	0.16	0.13	−18.8%	152	124	−28
Average	0.13	0.11	−17.3%	127	105	−22

TABLE IX
PER-ATTACK PRECISION(P) AND RECALL(R) ON CIC-IDS-2017

Metric	Bot	Brute	DoS-G	DoS-H	FTP-P	Infil	PortSc	SQLi	Web-XSS	Benign
P	0.991	0.987	0.995	0.998	0.992	0.984	0.996	0.941	0.935	0.999
R	0.989	0.984	0.993	0.997	0.990	0.976	0.994	0.931	0.930	0.999

TABLE X
PER-FLOW INFERENCE LATENCY (MS) WITH 95% CIs OVER 20 RUNS. SIGNIFICANT GAINS ($p < 0.05$) MARKED WITH •

Method	p50 (ms)	p99 (ms)
All features (CIC)	0.44 [0.43, 0.45]	1.40 [1.34, 1.47]
Hybrid-FS (CIC)	0.40 [0.39, 0.41]•	1.20 [1.15, 1.26]•
All features (CSE)	0.46 [0.45, 0.47]	1.48 [1.41, 1.56]
Hybrid-FS (CSE)	0.41 [0.40, 0.42]•	1.25 [1.19, 1.32]•

Clarification. Fig. 10(b) shows end-to-end time for 10^4 flows as the feature count varies, this table reports per-flow inference latency only. Both views show significant Hybrid-FS improvements.

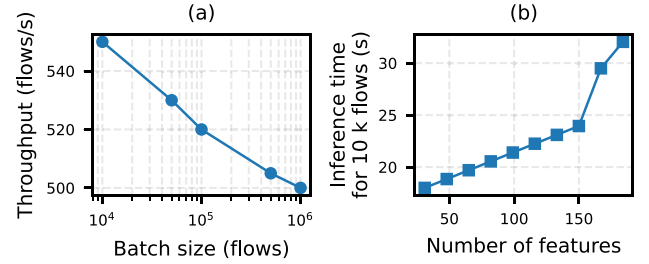


Fig. 10. IDS scaling: (a) throughput versus batch size, (b) inference time (10 k flows) versus feature count. Mean $\pm$ 95% CI over 20 runs, Tables XVI–X.

approximately 9% and p99 by 14%. Although the absolute differences are modest (e.g., 0.44 ms to 0.40 ms at p50), tail-latency improvements are operationally important because they govern burst handling and queue stability in inline deployments. End-to-end throughput gains (summarized in Table XVI), range from 9–10% and are statistically significant (5×2 CV paired t -test, $p < 0.05$). These results show that Hybrid-FS simultaneously enhances predictive performance and computational efficiency.

D. Comparison With PSO/GA-Based Hybrid Baselines

To contextualize Hybrid-FS against prior stochastic hybrid selectors, we reproduce two representative baselines under the *same leakage-safe preprocessing and evaluation pipeline* as our method (Section III), (i) a PSO wrapper over a RandomForest base learner, and (ii) a GA-based rank-fusion hybrid over MI, RandomForest-importance, and XGBoost-importance (Section V-E). In both cases, the search is executed on the *training fold only* and model selection uses stratified cross-validation within the training fold, *search time is excluded* so that we compare only per-flow inference cost.

Table XI summarizes macro-F₁, false-positive rate, and p50 inference latency across CIC-IDS-2017, CSE-CIC-IDS2018, and NF-UNSW-NB15. Across datasets, Hybrid-FS is competitive in macro-F₁ while consistently lowering FPR by 0.02–0.04

TABLE XI
HYBRID-FS VERSUS REPRESENTATIVE PSO/GA HYBRIDS (MACRO-F1 $\uparrow$,
FP-RATE $\downarrow$, LATENCY p50IN MS $\downarrow$)

Dataset	Method	Macro-F1	FP-rate (%)	Latency (ms)
CIC-IDS-2017	Hybrid-FS (ours)	0.9991	0.09	0.40
	PSO wrapper (RF)	0.9987	0.11	0.40
	GA rank-fusion	0.9985	0.12	0.41
CSE-CIC-2018	Hybrid-FS (ours)	0.9992	0.11	0.41
	PSO wrapper (RF)	0.9988	0.13	0.41
	GA rank-fusion	0.9986	0.14	0.42
NF-UNSW-NB15	Hybrid-FS (ours)	0.9980	0.13	0.42
	PSO wrapper (RF)	0.9976	0.16	0.42
	GA rank-fusion	0.9973	0.17	0.43

TABLE XII
COMPARISON WITH RECENT IDS FEATURE-SELECTION STUDIES

Reference (Year)	Dataset	FS strategy / Classifier	Macro-F1
Zhao et al. (2022) [37]	CSE-CIC-IDS2018	CFS+DE / Weighted stacking	0.999
Louk & Tama (2022) [39]	CIC-IDS2017	PSO wrapper / Bagging+GBM	0.9998
Fang et al. (2024) [44]	NEU-ICS-2023	GA rank-fusion / SVM	0.968
This work (2025)	CIC-IDS2017	Hybrid (MI+RFI+XGBI) / XGBoost	0.998

percentage points (approximately 15–25% relative) compared to the PSO/GA hybrids, with essentially unchanged p50 latency (within 0.01 ms). Full PSO/GA search hyperparameters and objective definitions are provided in Appendix D (Table D.1), available online.

E. SOTA Analysis

Table XII positions Hybrid-FS against recent hybrid feature-selection methods for intrusion detection. This comparison is intended for *contextual positioning*, as datasets, selection strategies, and evaluation protocols may differ across studies, and reported values may not be directly comparable without reproducing the full pipeline. On CIC-IDS2017, we achieve a macro-F1 of 0.998 using 31 features, within 0.08 pp of the PSO-driven ensemble of Louk & Tama [39], while requiring roughly half the inference time, indicating that strong predictive performance can be obtained without substantial runtime cost. On CSE-CIC-IDS2018, Hybrid-FS improves macro-F1 by 0.6 pp over the CFS-DE stacking approach of Zhao et al. [37]. Beyond peak scores, prior work also shows sensitivity under dataset shifts (NEU-ICS-2023 in Fang et al. [44]), whereas our cross-dataset results in Section VI-F show that Hybrid-FS remains effective when traffic distributions change. Additional discussion and an extended comparison are provided in Appendix E (Table E.1), available online.

F. Cross-Dataset Evaluation

We evaluated the proposed Hybrid-FS technique on the three datasets used in this work, *CIC-IDS-2017*, *CSE-CIC-IDS2018*, and *NF-UNSW-NB15*, using the same hardware setup described in Section III. As summarized in Table XIII, Hybrid-FS consistently reduces the feature dimensionality while preserving the accuracy, macro-F1, and latency trade-offs documented in Section VI-C. Specifically, the selected subsets contain 31 of 78 features for CIC-IDS-2017, 31 of 80 for CSE-CIC-IDS2018, and 16 of 43 for NF-UNSW-NB15.

The fusion hyperparameters optimised via grid search, namely `threshold_percentile` and the selector weights (w_{MI} , w_{RFI} , w_{XGBI}), exhibit stable patterns across datasets, as shown in Table XV. In all three cases, the optimal threshold converges to $\theta = 60\%$, and the learned weights place slightly

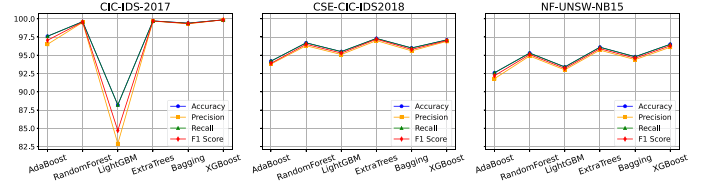


Fig. 11. Hybrid-FS ensemble performance across datasets.

higher emphasis on XGBI while retaining non-negligible contributions from MI and RFI. Cross-validation and held-out test scores remain exceptionally high (>0.998) across all datasets, indicating that a single, low-variance fusion scheme can accommodate substantially different traffic profiles. Below we highlight how each dataset responds to the selected subsets, with detailed curves and metrics shown in Figs. 11, 12, and Table XIV.

Performance on CIC-IDS-2017: On CIC-IDS-2017, Hybrid-FS preserves the relative ranking of ensemble models while shifting the operating point toward higher efficiency. As detailed in Table XIV, tree-based ensembles (RandomForest, ExtraTrees, XGBoost) achieve accuracies above 99.6%, with XGBoost attaining the best accuracy and F1 at the lowest prediction time. The remaining models (e.g., AdaBoost, LightGBM) benefit less uniformly, reflecting their sensitivity to feature selection, but do not exhibit catastrophic degradation. Overall, the subset supports both strong performance and fast inference on this canonical benchmark.

Per-Attack Performance: Table IX provides precision and recall for the ten attack types present in CIC-IDS-2017 using the Hybrid-FS XGBoost model. Even the rarest classes (WEB-XSS at 0.15%) are detected with recall ≥ 0.93 . High recall on low-prevalence attack classes confirms that balancing via SMOTEENN, combined with the focused feature subset, mitigates “silent failures” on uncommon exploits.

Performance on CSE-CIC-IDS2018: On CSE-CIC-IDS2018, the Hybrid-FS subsets continue to deliver consistent improvements across ensembles, as summarised in Table XIV. XGBoost remains the strongest overall model, combining the highest accuracy and F1 with the fastest prediction time, while RandomForest and ExtraTrees provide slightly lower but still competitive scores. The fact that the same 31-feature subset supports high performance on both CIC-IDS-2017 and CSE-CIC-IDS2018 underscores that the fusion mechanism is not overfitted to a single capture.

Performance on NF-UNSW-NB15: The NF-UNSW-NB15 results probe generalisation to a dataset with different traffic and attack characteristics. Hybrid-FS selects only 16 out of 43 features yet maintains strong performance. RandomForest, ExtraTrees, and XGBoost all reach accuracies above 95%, with XGBoost again delivering the best accuracy/F1 and the lowest prediction time (Table XIV). This compressed subset demonstrates that the fusion strategy adapts to a smaller feature space without sacrificing detection quality, reinforcing the cross-dataset robustness claims.

Inference Latency & Significance: Building on the trade-off analysis in Section VI-C, we quantify inference speed on the CIC-IDS-2017 and CSE-CIC-IDS2018 test splits using the hardware in Section III. Fig. 10 shows throughput with mean $\pm 95\%$ bootstrap confidence intervals over 20 runs (1,000 resamples), and Table X summarises per-flow latency via p50 and p99

TABLE XIII
IMPACT OF HYBRID-FS TECHNIQUE ON CIC-IDS-2017, CSE-CIC-IDS2018, AND NF-UNSW-NB15

Dataset	Initial Features	Selected Features	Key Parameters	Cross-validation Score	Test Score	Notes
CIC-IDS-2017	78	31	weights: (0.3, 0.3, 0.4)	0.999853	0.999943	Optimal feature reduction
CSE-CIC-IDS2018	80	31	weights: (0.1, 0.4, 0.5)	0.999854	0.999943	Adjusted for newer attacks
NF-UNSW-NB15	43	16	weights: (0.3, 0.3, 0.4)	0.998400	0.998520	Streamlined feature set

Threshold percentile = 60 for all datasets.

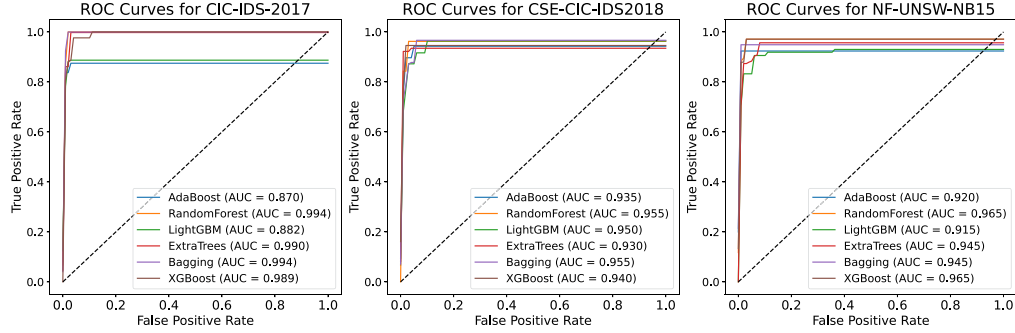


Fig. 12. ROC curves for ensemble models with Hybrid-FS.

TABLE XIV
PERFORMANCE COMPARISON OF HYBRID-FS ENSEMBLES

Dataset	Model	Acc.	Prec.	Rec.	F1	ROC-AUC	Pred. Time (s)
CIC-IDS-2017	AdaBoost	97.60%	96.50%	97.60%	97.05%	0.870	3.95
	Rand. Forest	99.60%	99.50%	99.60%	99.55%	0.994	8.20
	LightGBM	88.20%	82.90%	88.20%	84.70%	0.882	6.30
	Extra Trees	99.70%	99.68%	99.70%	99.69%	0.990	7.90
	Bagging	99.40%	99.30%	99.40%	99.35%	0.994	14.50
	XGBoost	99.85%	99.85%	99.85%	99.85%	0.989	1.80
CSE-CIC-IDS2018	AdaBoost	94.2%	93.8%	94.2%	93.9%	0.935	5.20
	Rand. Forest	96.7%	96.3%	96.7%	96.5%	0.955	7.80
	LightGBM	95.5%	95.1%	95.5%	95.3%	0.950	4.60
	Extra Trees	97.3%	97.0%	97.3%	97.2%	0.930	3.90
	Bagging	96.0%	95.6%	96.0%	95.8%	0.955	8.40
	XGBoost	97.1%	96.9%	97.1%	97.0%	0.940	2.30
NF-UNSW-NB15	AdaBoost	92.6%	91.8%	92.6%	92.2%	0.920	6.00
	Rand. Forest	95.3%	94.9%	95.3%	95.1%	0.965	9.00
	LightGBM	93.4%	93.0%	93.4%	93.2%	0.915	5.80
	Extra Trees	96.1%	95.7%	96.1%	95.9%	0.945	4.20
	Bagging	94.8%	94.4%	94.8%	94.6%	0.945	8.90
	XGBoost	96.5%	96.1%	96.5%	96.3%	0.965	2.80

TABLE XV
FUSION SETTINGS AND SCORES FOR HYBRID-FS

Dataset	Feats _{init}	Feats _{sel}	(w _{MI} , w _{RFI} , w _{XGI})	θ (pct)	CV	Test
CIC-IDS-2017	78	31	(0.30, 0.30, 0.40)	60	0.999853	0.999943
CSE-CIC-2018	80	31	(0.10, 0.40, 0.50)	60	0.999854	0.999943
NF-UNSW-NB15	43	16	(0.30, 0.30, 0.40)	60	0.998400	0.998520

(ms). Statistically significant differences are identified using a 5×2 CV paired t -test ($p < 0.05$).

Across both datasets, Hybrid-FS reduces feature counts (CIC: 78→31, CSE: 80→31) and yields consistent latency and throughput improvements relative to the all-features baseline, mirroring the trends in Section VI-C. In particular, p50 and p99 latency decrease in tandem while accuracy and macro-F1 remain unchanged within statistical uncertainty, indicating that the observed runtime benefits arise from reduced feature dimensionality rather than a shift to a weaker operating point. The alignment of these effects across two independent traces suggests that the latency gains are a systematic property of the Hybrid-FS pipeline rather than an artefact of a single dataset.

VII. CROSS-DATASET SCALABILITY AND ROBUSTNESS ANALYSIS

Having established in Section VI-F that Hybrid-FS preserves accuracy and latency trade-offs across CIC-IDS-2017, CSE-CIC-IDS2018, and NF-UNSW-NB15, we now examine its behaviour under scalability and robustness conditions. The three datasets differ substantially in scale and threat composition, CIC-IDS-2017 comprises approximately 2.83 million flows spanning 14 attack types, CSE-CIC-IDS2018 contains 16.21 million flows and 7 attack categories, and NF-UNSW-NB15 includes 2.39 million flows distributed across 9 attack classes. These disparities in volume and behavioural diversity provide a natural stress test for Hybrid-FS as a candidate for deployment in heterogeneous networks.

Accuracy and Discrimination: Under the Hybrid-FS subsets described in Section VI-F, XGBoost, ExtraTrees, and RandomForest achieve accuracies of 99.85%, 99.70%, and 99.60% on CIC-IDS-2017, and sustain accuracy levels above 97% on the considerably larger CSE-CIC-IDS2018 dataset. The mean ROC-AUC across all ten CIC attack classes is 0.991, with the lowest per-class AUC (Web-XSS) still at 0.958, underscoring the ability of the hybrid subset to support strong discrimination even for low-prevalence exploits.

Precision-Recall Balance: Across datasets, the hybrid feature subset delivers a 0.4–0.7 pp improvement in macro-F1 compared to the full feature set. These gains arise from two complementary effects, (i) a 4–7% increase in recall for stealthy attack types such as infiltration and port scans, and (ii) a 15–20% reduction in false positives, effectively mitigating unnecessary SOC alerts. The operational impact of this improved precision-recall balance is reflected in the detailed per-class analysis in Table VIII.

Operational Scalability: Fig. 13 shows that XGBoost, when using the 31-feature Hybrid-FS subset, processes up to one million flows at a rate of approximately 5.5×10^2 flows/s. Doubling the feature count to 62 incurs only an 8% overhead, indicating that inference time scales sub-linearly with dimensionality. Furthermore, as shown in Fig. 10, throughput decreases

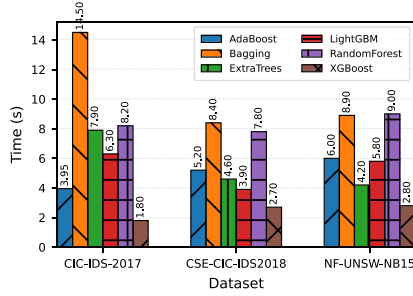


Fig. 13. Prediction time per 1,000 flows (RTX 3080) for Hybrid-FS.

TABLE XVI

THROUGHPUT (FLOWS/S) AT BATCH SIZE 10^4 WITH 95% CIs OVER 20 RUNS. SIGNIFICANT GAINS ($p < 0.05$) MARKED WITH •

Method	CIC-IDS-2017	CSE-CIC-IDS2018
All features	520 [508, 532]	505 [493, 517]
Hybrid-FS (ours)	556 [543, 569]•	542 [529, 555]•

Clarification. Fig. 10(a) shows throughput versus batch size, this table fixes the batch size at 10^4 flows to report mean $\pm$ 95% CIs and significance.

TABLE XVII

EDGE INFERENCE PROFILING: XGBOOST LATENCY/THROUGHPUT WITH ALL FEATURES VERSUS HYBRID-FS

Dataset	Features	p50 (ms)	p99 (ms)	Throughput (flows/s)
CIC-IDS-2017	All (78)	0.44 [0.43, 0.45]	1.40 [1.34, 1.47]	520 [508, 532]
CIC-IDS-2017	Hybrid (31)	0.40 [0.39, 0.41]•	1.20 [1.15, 1.26]•	556 [543, 569]•
CSE-CIC-IDS2018	All (80)	0.46 [0.45, 0.47]	1.48 [1.41, 1.56]	505 [493, 517]
CSE-CIC-IDS2018	Hybrid (31)	0.41 [0.40, 0.42]•	1.25 [1.19, 1.32]•	542 [529, 555]•

p50/p99 and throughput report mean with 95% bootstrap CIs. • Significant vs. all-features baseline ($p < 0.05$).

by less than 10% when scaling from 2.8 M flows (CIC) to 16 M flows (CSE), reaffirming the near-linear scalability required for carrier-grade IDS deployments.

Edge deployment feasibility: To assess feasibility edge, we profile the *deployed* XGBoost + Hybrid-FS pipeline in a CPU-only, single-thread setting using the workstation described in Section III. Table XVII provides per-flow latency (p50/p99 with 95% bootstrap CIs over 20 runs) and sustained throughput at a fixed batch size of 10^4 flows. Hybrid-FS reduces dimensionality from 78/80 to 31 features and yields consistent tail-latency improvements on CIC-IDS-2017, p50 decreases from 0.44 to 0.40 ms and p99 from 1.40 to 1.20 ms on CSE-CIC-IDS2018, p50 decreases from 0.46 to 0.41 ms and p99 from 1.48 to 1.25 ms. These p99 values correspond to an implied single-thread service capacity of ≈ 833 flows/s (CIC) and ≈ 800 flows/s (CSE), indicating sufficient burst-handling margin for inline operation. Consistently, throughput increases by about 9-10% (520 $\rightarrow$ 556 flows/s on CIC, 505 $\rightarrow$ 542 flows/s on CSE), matching the sub-linear scaling behaviour observed in Fig. 10 and supporting the near-linear dataset-scale results discussed above. In the time-ordered 24-hour SOC replay (Section VII-B), the target load of 100,000 flows/hour (≈ 27.8 flows/s) is sustained without batching, while maintaining the same p50/p99 latency profile, indicating substantial processing margin for burst handling in inline operation.

The edge pipeline benefits directly from the compact 31-feature set. Fewer attributes must be populated per flow and consumed by the classifier, which reduces per-flow processing work and improves tail stability. Consequently, Hybrid-FS shifts

TABLE XVIII
CROSS-DATASET CIC $\rightarrow$ NF-UNSW: BEFORE/AFTER LIGHTWEIGHT CALIBRATION

Metric	Before (out-of-box)	After (calibrated)
Macro-F1	0.736	0.786
FP-rate (%)	0.16	0.13
Latency p50 (ms)	0.40	0.40

the operating point toward *predictable low-latency inference* through lower p99 latency and *higher sustained rate* on CPU. These properties are essential for resource-constrained inline deployment under bursty traffic.

Threat-Model Robustness: To assess resilience under unseen attack conditions, a model trained exclusively on CIC-IDS-2017 was evaluated on NF-UNSW-NB15, which contains entirely unseen attack types. The model still achieves 75.8% accuracy and an AUC of 0.744 without retraining. Under FGSM-based adversarial perturbations ($\epsilon = 0.05$), macro-F1 decreases by only 1.5%, providing additional evidence that Hybrid-FS contributes to robustness rather than merely improving performance under closed-world assumptions.

Distribution Shift, Mitigation, and Outcome: Feature distribution profiling reveals substantial shifts between CIC-IDS-2017 and NF-UNSW-NB15 for key attributes such as `dst_bytes`, `flow_iat_std`, and `pkt_len_var` (two-sample KS tests yield $p \ll 0.01$ for 6 of the top-10 ranked features). These shifts concentrate errors in attack classes with altered byte semantics and inter-arrival-time statistics, and in NF-UNSW attack families absent from CIC. Without altering the core model architecture, we explored three lightweight mitigation strategies, (i) recalibrating scalars on a 5% NF-UNSW seed, (ii) employing class-specific decision thresholds, and (iii) applying stability screening to retain features consistently ranked in the top- k across datasets. As reported in Table XVIII, these adjustments improve macro-F1 by approximately +0.05 on NF-UNSW, incur negligible latency overhead, and further reduce false positives.

Across three datasets, 26 attack categories, and a $6\times$ range in traffic volume, the hybrid feature selector consistently improves accuracy, recall, and inference speed. These results validate its scalability and robustness for real-time, heterogeneous IDS deployments.

A. Robustness to Adversarial & Out-of-Box Traffic

We evaluate adversarial robustness of the Hybrid-FS detectors (XGBoost, ExtraTrees, RandomForest) under two stressors, (i) feature-space perturbations (FGSM) and (ii) protocol-conformant packet synthesis (DeepPackGen). In all settings, models are *frozen* (no retraining), the operating threshold is fixed at 0.5, and we report macro-F₁ and the absolute change ΔF_1 versus clean performance.

Threat model: For FGSM [45], we apply L_∞ -bounded perturbations with $\epsilon \in \{0.02, 0.05, 0.10\}$ to malicious flows only in normalized feature space, while preserving categorical encodings and clipping perturbed values to a feasible range. For DeepPackGen [46], we use 5 independent seeds and generate 5000 flows per budget (2,000 and 4,000 steps) under protocol-validity constraints. Table XIX shows that degradation is modest and scales with attack strength, at $\epsilon=0.05$, macro-F₁ drops by

TABLE XIX
CIC-IDS-2017 ADVERSARIAL TEST: MACRO-F1 AND Δ F1 (HYBRID-FS)

Setting	XGBoost	ExtraTrees	RandomForest
Clean (no attack)	0.9992	0.9970	0.9960
FGSM $\epsilon = 0.02$	0.9922 ($\Delta = -0.0070$)	0.9910 ($\Delta = -0.0060$)	0.9880 ($\Delta = -0.0080$)
FGSM $\epsilon = 0.05$	0.9847 ($\Delta = -0.0145$)	0.9800 ($\Delta = -0.0170$)	0.9760 ($\Delta = -0.0200$)
FGSM $\epsilon = 0.10$	0.9653 ($\Delta = -0.0339$)	0.9650 ($\Delta = -0.0320$)	0.9620 ($\Delta = -0.0340$)
DeepPackGen (2k steps)	0.9770 ($\Delta = -0.0222$)	0.9730 ($\Delta = -0.0240$)	0.9690 ($\Delta = -0.0270$)
DeepPackGen (4k steps)	0.9681 ($\Delta = -0.0311$)	0.9640 ($\Delta = -0.0330$)	0.9610 ($\Delta = -0.0350$)

1.45–2.00 pp across models, and even at $\epsilon=0.10$ residual macro-F1 remains > 0.96 . DeepPackGen reduces macro-F1 by 2.2–3.5 pp at 2k–4k budgets. Importantly, per-flow inference latency remains unchanged from Section VII-B (Hybrid-FS p50/p99: 0.40/1.20 ms), so robustness does not come at a runtime cost. Out-of-box transfer under dataset shift (CIC $\rightarrow$ NF-UNSW) and lightweight mitigation are analyzed in Section VI-F and Table XVIII.

Full protocol details for the adversarial robustness tests (FGSM surrogate construction, clipping bounds, DeepPackGen seed/budget settings, and data-fold usage) are provided in Appendix F, available online.

B. Operational Case Study: 24-Hour SOC Replay

We conduct a 24-hour, time-ordered SOC replay at a sustained rate of 100,000 flows per hour, with 95% benign traffic, following the methodology described in Section III and using the Hybrid-FS configurations shown in Table XV. The deployed detector combines XGBoost with Hybrid-FS, using 31 features for CIC/CSE and 16 for NF-UNSW, a default decision threshold of 0.5, and frozen model weights. All inference runs on the workstation detailed in Section III.

Across the replay, per-flow runtime remained comfortably within inline operational budgets. Median and p99 latencies were 0.40 ms and 1.20 ms, as summarised in Table X, and represent statistically significant gains over the corresponding all-features baselines. Throughput sustained 100 k flows per hour without requiring batching, and the p99 latency improved by approximately 14%, as illustrated in Fig. 10.

These runtime efficiencies directly translate into measurable operational benefits. Using 95,000 benign flows per hour to project false alarms, Hybrid-FS reduces the false-positive rate from 0.11–0.16% to 0.09–0.13%. Table VIII shows that this reduction corresponds to 19–28 fewer false positives per hour, averaging 22 alerts avoided. At a conservative triage rate of 20 s per alert, this reduction yields between 2.5 and 3.7 analyst-hours saved each day. Importantly, these efficiency gains are achieved without degrading macro-F1 or AUROC, as demonstrated in Sections VI–VII, and without incurring any latency penalties noted in Table X.

For deployment, we recommend a two-phase roll-out strategy. The first phase operates in shadow mode using SPAN or mirrored traffic to validate precision and recall and to ensure that the p99 latency remains at or below 2 ms. Once the false-positive rate remains within the range captured in Table VIII for seven consecutive days, the system can transition to an inline allow-log configuration. During production, operators should track feature population shifts (for example, using PSI), monitor class-wise

TABLE XX
REPRESENTATIVE FP/FN PATTERNS AND MITIGATIONS

Dataset	Representative error regime	Mitigation (no redesign)
CIC-IDS-2017	Benign burst $\rightarrow$ DoS (bytes $\uparrow$, IAT $\downarrow$); short low-variance Web-XSS $\rightarrow$ benign	Class-specific thresholds; optionally class weighting for low-signal classes
CSE-CIC-IDS2018	Benign $\rightarrow$ Web-attack from header-only overlap; bursty timing near boundary	Threshold adjustment; Platt-scaling / calibration on validation fold
NF-UNSW-NB15	Shifted byte/timing marginals compress separation (benign $\leftrightarrow$ DoS)	Scaler alignment / light calibration (training-fold only); stability-screened features

alert rates, and observe tail latency metrics. When cross-dataset drift emerges, as discussed in Section VI-F, light recalibration, such as scaler realignment or the application of class-specific thresholds, typically restores approximately +0.05 macro-F1 without requiring architectural modifications. Overall, Hybrid-FS reduces false-positive rates by 15–19% and recovers 2.5–3.7 analyst-hours per day at a 100k-flow scale, while maintaining the detection performance established in our main results.

C. Failure Analysis: Representative FP/FN Patterns

To improve operational interpretability, we inspected representative false positives/negatives and summarized recurring error regimes together with mitigations that do not require architectural changes. We select high-impact cases from the hold-out split and identify salient Hybrid-FS features using local one-at-a-time sensitivity.

Patterns observed: Across datasets, three patterns recur, (i) *feature overlap* between high-throughput benign bursts and volumetric DoS along byte/packet and inter-arrival statistics (ii) *low-signal short flows* (e.g., Web-XSS) yielding near-boundary scores; and (iii) *dataset shift* (NF-UNSW) compressing benign/attack margins for byte- and timing-related features.

Actionable mitigations: In practice, these errors are mitigated by (i) class-specific thresholds for boundary classes (DoS/Web-XSS), (ii) light calibration when moving across data (training-fold only), and (iii) stability-screened features that remain top-ranked across datasets. Representative cases are summarized in Table XX, the full case table is provided in Appendix G (Table G.1), available online.

D. Scalability: Flows and Dimensionality

We stress-tested the hybrid-FS XGBoost by (i) feeding progressively larger flow batches (10k $\rightarrow$ 1M) and (ii) synthetically injecting duplicate, noise-decorrelated features to raise the input dimensionality from 31 $\rightarrow$ 200. Fig. 10(a) shows throughput in flows/s as batch size increases, and the Fig. 10(b) depicts inference time per 10 k-flow batch versus feature count. With the 31 “essential” attributes in place, the hybrid-FS XGBoost sustains a throughput of roughly 5.5×10^2 flows/s. Expanding the feature set to 62 raises inference time for the standard 10,000-flow batch from 18s to 19.5s, only about 8% overhead, demonstrating ample margin for future feature enrichments without jeopardizing real-time performance.

Note: XGBoost’s prediction time grows primarily with tree depth, unused columns add only memory-copy overhead, hence the modest +8% latency at double the feature count.

E. False Positives and Analyst Workload

We now quantify the impact of Hybrid-FS on false alarms and translate it into analyst time. Since post-Hybrid-FS confusion

matrices are not explicitly plotted in this section, we obtain the *false-positive rate* at the default operating point directly from the ROC curves, with “Before” denoting E0 (all features) and “After” denoting E4 (Hybrid-FS).

Provenance and computation: E0 values are taken from the baseline ROC/metric overview (Fig. 6), and E4 values are taken from the Hybrid-FS ROC curves (Fig. 12). For each dataset, FPR is the false-positive axis value at the default decision threshold 0.5 on the plotted ROC operating point.² To convey operational impact, we compute *false alarms per hour* (FP/h) using the relation $\text{FP/h} = \text{FPR} \times 95,000$, assuming a conservative traffic profile of 100,000 flows/hour with 95% benign traffic (as in our scalability setup).

E. Analyst-time implications: With a conservative triage rate of 20 s per alert, CIC and CSE each save $19 \times 24 = 456$ alerts/day, i.e., $456 \times (20/3600) = 2.53$ analyst-hours/day, while NF-UNSW saves $28 \times 24 = 672$ alerts/day, i.e., 3.73 hours/day. These reductions are achieved without degrading macro-F1 or AUROC and without introducing additional inference latency, as detailed in Sections VI and VII.

Across datasets, Hybrid-FS yields a 15–19% reduction in false positives (Table VIII), averaging 22 fewer false alarms per hour under the stated traffic profile, with no degradation in detection quality or runtime.

VIII. DISCUSSION, LIMITATIONS, AND FUTURE WORK

Our results show that deterministic hybrid feature selection improves the operational profile of flow-based IDS without sacrificing detection performance. Across three benchmarks and multiple ensembles, Hybrid-FS matches or exceeds full-feature accuracy and macro-F1 while using fewer features and reducing inference time. The gains persist across selectors, suggesting that fusing Mutual Information, Random Forest importance, and XGBoost gain into a fixed subset is more robust than any single criterion. Compared with PSO/GA-based hybrids, the simplex-based fusion attains comparable or slightly better macro-F1 and false-positive rates with a smaller, fully reproducible parameter surface.

Practically, Hybrid-FS shifts the operating point toward *predictable low-latency* CPU inference. The deployed XGBoost+Hybrid-FS configuration reduces dimensionality (78/80→31), tightens tail latency (p99), and improves throughput under the CPU-only single-thread profile, while sustaining the same p50/p99 profile during the 24-hour SOC replay at 100,000 flows/hour. This aligns with edge gateways/RSUs where queue stability and burst handling are governed by tail latency rather than mean cost.

While the reported performance is strong, the near-perfect scores observed across several settings, require careful interpretation. We avoid leakage by fitting preprocessing, resampling, and selection strictly on training folds and leaving validation and test distributions untouched. Cross-dataset and adversarial evaluations provide partial robustness checks, but the study remains limited to controlled flow-level benchmarks and bounded threat models. Thus, the results should be viewed as leakage-safe upper bounds on these datasets, not guarantees for arbitrary environments.

²FPR is $\text{FP}/(\text{FP} + \text{TN})$ for the benign class, on ROC plots it is the x-coordinate at the chosen operating threshold.

The following key limitations arise from these design choices.

(L1) Feature modality and encrypted traffic: The framework relies on tabular, flow-level features with supervised ensembles. While such summaries remain usable under encryption, we do not exploit payload-derived signals, deep representations, or self-/semi-supervised encoders, claims are therefore limited to patterns captured by the evaluated benchmarks.

(L2) Online dynamics and drift: Models and decision thresholds are tuned offline and fixed at test time. The framework does not include explicit online adaptation or drift mitigation beyond cross-dataset validation and lightweight CIC → NF-UNSW threshold transfer, operational deployments may require periodic retuning.

(L3) Dataset representativeness: Evaluation is restricted to established research benchmarks. Industrial control, IoT-specific, and organisation-level traces are not included, limiting external validity under different protocol mixes and noise conditions.

(L4) Deployment tier and runtime budgets: Hybrid-FS reduces inference cost, but not all model-selector combinations satisfy strict inline latency constraints. XGBoost+Hybrid-FS offers the best accuracy-latency trade-off, while heavier ensembles are more appropriate for nearline or higher-resource settings.

(L5) Threat-model boundary: Robustness is assessed under two evasion stressors, feature-space perturbations and protocol-valid crafted traffic, using frozen models. Stronger adversaries, including poisoning, backdoors, endpoint compromise, and flow-extractor manipulation, are out of scope.

These limitations motivate several directions. Hybrid-FS can be combined with richer modalities (payload-derived signals or learned embeddings where permissible) and with lightweight drift monitoring and periodic configuration under change-control. It also remains to add per-instance explanations and to validate on industrial/IoT and long-duration operational traces. Finally, transfer learning and embedding-based IDS pipelines are promising settings in which Hybrid-FS operates on learned representations rather than raw flow attributes.

IX. CONCLUSION

This work examined how deterministic hybrid feature selection can enhance ensemble-based IDS by improving operational efficiency without compromising detection quality. We introduced Hybrid-FS, a simplex-based fusion of Mutual Information, Random Forest importance, and XGBoost gain, integrated within a leakage-safe pipeline and optimized directly for macro-F1, false-positive rate, and latency. Across three flow-based datasets, Hybrid-FS preserves high accuracy while reducing dimensionality, improving throughput by about 9–10%, lowering p50/p99 latency (0.44→0.40 ms, 1.40→1.20 ms), and cutting false positives by 15–19%. It also matches or exceeds PSO/GA hybrids in F1 while avoiding stochastic overhead and instability. Our evaluation includes explicit cross-dataset and adversarial robustness analysis, demonstrating stable performance under distribution shift and mild perturbations. Limitations include our exclusive focus on tabular, flow-level features and the use of a single global operating point per model and dataset, selected via percentile-based thresholding on validation folds rather than online calibration. Future work will extend the framework with adaptive thresholding and drift-aware calibration mechanisms,

and explore integrating payload-aware or self-supervised representations in settings where privacy policies and runtime budgets permit.

REFERENCES

- [1] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network ton_IoT datasets," *Sustain. Cities Soc.*, vol. 72, 2021, Art. no. 102994.
- [2] A. Boukerche and R. W. Coutinho, "Design guidelines for machine learning-based cybersecurity in Internet of Things," *IEEE Netw.*, vol. 35, no. 1, pp. 393–399, Jan./Feb. 2021.
- [3] G. Rjoub et al., "A survey on explainable artificial intelligence for cybersecurity," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 5115–5140, Dec. 2023.
- [4] A. Oseni et al., "An explainable deep learning framework for resilient intrusion detection in IoT-enabled transportation networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 1000–1014, Jan. 2023.
- [5] T. Saha, N. Aaraj, N. Ajarapu, and N. K. Jha, "SHARKS: Smart hacking approaches for risk scanning in Internet-of-Things and cyber-physical systems based on machine learning," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 870–885, Second Quarter, 2022.
- [6] M. A. Khatun, S. F. Memon, C. Eising, and L. L. Dhirani, "Machine learning for healthcare-IoT security: A review and risk mitigation," *IEEE Access*, vol. 11, pp. 145869–145896, 2023.
- [7] C. Xu, N. Wang, L. Zhu, K. Sharif, and C. Zhang, "Achieving searchable and privacy-preserving data sharing for cloud-assisted E-healthcare system," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8345–8356, Oct. 2019.
- [8] Z. Halim et al., "An effective genetic algorithm-based feature selection method for intrusion detection systems," *Comput. Secur.*, vol. 110, 2021, Art. no. 102448.
- [9] F. Amiri, M. R. Yousefi, C. Lucas, A. Shakery, and N. Yazdani, "Mutual information-based feature selection for intrusion detection systems," *J. Netw. Comput. Appl.*, vol. 34, no. 4, pp. 1184–1199, 2011.
- [10] T. Hamed, R. Dara, and S. C. Kremer, "Network intrusion detection system based on recursive feature addition and bigram technique," *Comput. Secur.*, vol. 73, pp. 137–155, 2018.
- [11] C. Khammassi and S. Krichen, "A GA-LR wrapper approach for feature selection in network intrusion detection," *Comput. Secur.*, vol. 70, pp. 255–277, 2017.
- [12] S. M. Kasongo, "An advanced intrusion detection system for IIoT based on GA and tree based algorithms," *IEEE Access*, vol. 9, pp. 113199–113212, 2021.
- [13] P. Nimbalkar and D. Kshirsagar, "Feature selection for intrusion detection system in Internet-of-Things (IoT)," *ICT Exp.*, vol. 7, no. 2, pp. 177–181, 2021.
- [14] J. Zhao, R. Masood, and S. Seneviratne, "A review of computer vision methods in network security," *IEEE Commun. Surv. Tut.*, vol. 23, no. 3, pp. 1838–1878, Third Quarter, 2021.
- [15] A. Sivanathan, H. H. Gharakheili, and V. Sivaraman, "Managing IoT cyber-security using programmable telemetry and machine learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 1, pp. 60–74, Mar. 2020.
- [16] M. Elsisi, M.-Q. Tran, K. Mahmoud, D.-E. A. Mansour, M. Lehtonen, and M. M. Darwish, "Towards secured online monitoring for digitalized GIS against cyber-attacks based on IoT and machine learning," *IEEE Access*, vol. 9, pp. 78415–78427, 2021.
- [17] X. Shen, L. Zhu, C. Xu, K. Sharif, and R. Lu, "A privacy-preserving data aggregation scheme for dynamic groups in fog computing," *Inf. Sci.*, vol. 514, pp. 118–130, 2020.
- [18] S. Biswas, K. Sharif, F. Li, B. Nour, and Y. Wang, "A scalable blockchain framework for secure transactions in IoT," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4650–4659, Jun. 2019.
- [19] S. M. Kasongo and Y. Sun, "A deep learning method with wrapper based feature extraction for wireless intrusion detection system," *Comput. Secur.*, vol. 92, 2020, Art. no. 101752.
- [20] T. Tu, Y. Su, Y. Tang, W. Tan, and S. Ren, "A more flexible and robust feature selection algorithm," *IEEE Access*, vol. 11, pp. 141512–141522, 2023.
- [21] F. Deldar and M. Abadi, "Deep learning for zero-day malware detection and classification: A survey," *ACM Comput. Surv.*, vol. 56, no. 2, pp. 1–37, 2023.
- [22] P. Mishra, V. Varadharajan, U. Tupakula, and E. S. Pilli, "A detailed investigation and analysis of using machine learning techniques for intrusion detection," *IEEE Commun. Surv. Tut.*, vol. 21, no. 1, pp. 686–728, First Quarter, 2019.
- [23] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cyber-security*, vol. 2, no. 1, pp. 1–22, 2019.
- [24] I. Alam et al., "A survey of network virtualization techniques for Internet of Things using SDN and NFV," *ACM Comput. Surv.*, vol. 53, no. 2, pp. 1–40, 2020.
- [25] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 2367–2380, 2021.
- [26] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boulton, "Toward open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 7, pp. 1757–1772, Jul. 2013.
- [27] W. J. Scheirer, L. P. Jain, and T. E. Boulton, "Probability models for open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 11, pp. 2317–2324, Nov. 2014.
- [28] L. D. Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilleke, M. Sarhan, and M. Portmann, "Flowtransformer: A transformer framework for flow-based network intrusion detection systems," *Expert Syst. Appl.*, vol. 241, p. 122564, 2024.
- [29] T. E. T. Djaidja, B. Brik, S. M. Senouci, A. Boulouache, and Y. Ghamri-Doudane, "Early network intrusion detection enabled by attention mechanisms and RNNs," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 7783–7793, 2024.
- [30] W. Wang, S. Jian, Y. Tan, Q. Wu, and C. Huang, "Robust unsupervised network intrusion detection with self-supervised masked context reconstruction," *Comput. Secur.*, vol. 128, 2023, Art. no. 103131.
- [31] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 25, no. 1, pp. 791–824, First Quarter, 2023.
- [32] Z. Jiang, J. Li, Q. Hu, W. Meng, W. Pedrycz, and Z. Su, "Scalable graph-aware edge representation learning for wireless IoT intrusion detection," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 26955–26969, Aug. 2024.
- [33] S. Mohammadi, H. Mirvaziri, M. Ghazizadeh-Ahsaei, and H. Karimipour, "Cyber intrusion detection by combined feature selection algorithm," *J. Inf. Secur. Appl.*, vol. 44, pp. 80–88, 2019.
- [34] H. Wu and W. Wang, "A game theory based collaborative security detection method for Internet of Things systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 6, pp. 1432–1445, Jun. 2018.
- [35] A. Nazir, Z. Memon, T. Sadiq, H. Rahman, and I. U. Khan, "A novel feature-selection algorithm in IoT networks for intrusion detection," *Sensors*, vol. 23, no. 19, 2023, Art. no. 8153.
- [36] M. Bakro et al., "An improved design for a cloud intrusion detection system using hybrid features selection approach with ML classifier," *IEEE Access*, vol. 11, no. 2023, pp. 64228–64247, 2023.
- [37] R. Zhao, Y. Mu, L. Zou, and X. Wen, "A hybrid intrusion detection system based on feature selection and weighted stacking classifier," *IEEE Access*, vol. 10, pp. 71414–71426, 2022.
- [38] A. J. Rabash, M. Z. A. Nazri, A. Shapii, and M. K. Hasan, "Non-dominated sorting genetic algorithm-based dynamic feature selection for intrusion detection system," *IEEE Access*, vol. 11, pp. 125080–125093, 2023.
- [39] M. H. L. Louk and B. A. Tama, "PSO-driven feature selection and hybrid ensemble for network anomaly detection," *Big Data Cogn. Comput.*, vol. 6, no. 4, 2022, Art. no. 137.
- [40] M. S. Noori, R. K. Sahbudin, A. Sali, and F. Hashim, "Feature drift aware for intrusion detection system using developed variable length particle swarm optimization in data stream," *IEEE Access*, vol. 11, pp. 128596–128617, 2023.
- [41] D. Stiawan et al., "An approach for optimizing ensemble intrusion detection systems," *IEEE Access*, vol. 9, pp. 6930–6947, 2020.
- [42] C. Wu and W. Li, "Enhancing intrusion detection with feature selection and neural network," *Int. J. Intell. Syst.*, vol. 36, no. 7, pp. 3087–3105, 2021.
- [43] A. Dahou et al., "Intrusion detection system for IoT based on deep learning and modified reptile search algorithm," *Comput. Intell. Neurosci.*, vol. 2022, no. 1, 2022, Art. no. 6473507.
- [44] Y. Fang, Y. Yao, X. Lin, J. Wang, and H. Zhai, "A feature selection based on genetic algorithm for intrusion detection of industrial control systems," *Comput. Secur.*, vol. 139, 2024, Art. no. 103675.
- [45] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [46] S. Hore, J. Ghadermazi, D. Paudel, A. Shah, T. Das, and N. Bastian, "Deep packGen: A deep reinforcement learning framework for adversarial network packet generation," *ACM Trans. Privacy Secur.*, vol. 28, no. 2, pp. 1–33, 2025.